CSC-204: T20WC Cricket Simulator

TEAM MEMBERS

Name	Enrollment No.	Contributions
Ankush Kumar Singh	24112018	Deadlock handling, PPT, PDF, Minor coding segments, Control flow visualization
Dhruv Jain	24114033	Major coding segments, Introduced parallelism, Handling version control
Jeel Amit Gandhi	24114046	Minor coding segments, Data scraping, Proposed an alternative solution to a problem
A.V. Tanish Reddy	24114014	Deadlock handling, Minor coding segments, Handled synchronization among batsmen
Ankit Singh	24114013	Minor coding and logic inputs

CONSIDERATIONS / SYSTEM ARCHITECTURE

We model a simulator for a T20 cricket match (namely between India (IND) and New Zealand (NZ)). Our goal with the implementation was to ensure that the simulation is both realistic while still being implementable using the functionalities of POSIX pthreads. Also, we ensure that the events of the simulation are truly processed in **real-time**; we have tried to avoid a predetermined sequential execution order wherever possible. We list our primary considerations below:

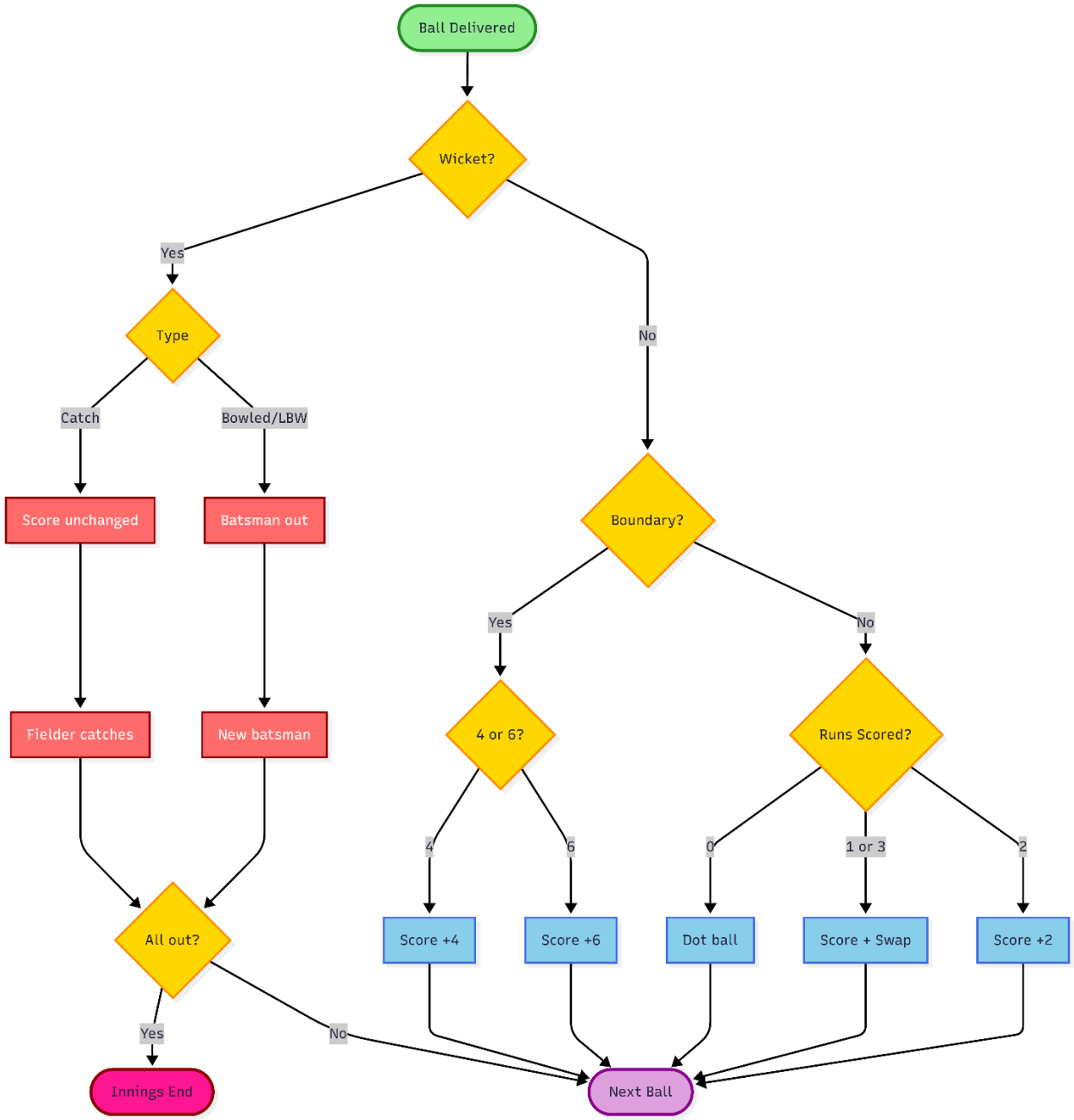
- Each team has 11 members which are spawned as separate dedicated threads , each created by a central coordinating umpire thread.
- Each player has unique statistics that determine how “good” of a batsman or bowler they are.
- At any instance, 2 batsmen threads are active and occupy the crease, and 1 bowler delivers balls.
- A batsman cannot process a delivery bowled by the bowler until it has fully simulated the events of the previous delivery.
- The fielder threads remain blocked and respond in real-time to the BALL_HIT signal. They awaken only in the case a run-out or catch occurs.

CONTROL FLOW OF THE SIMULATION

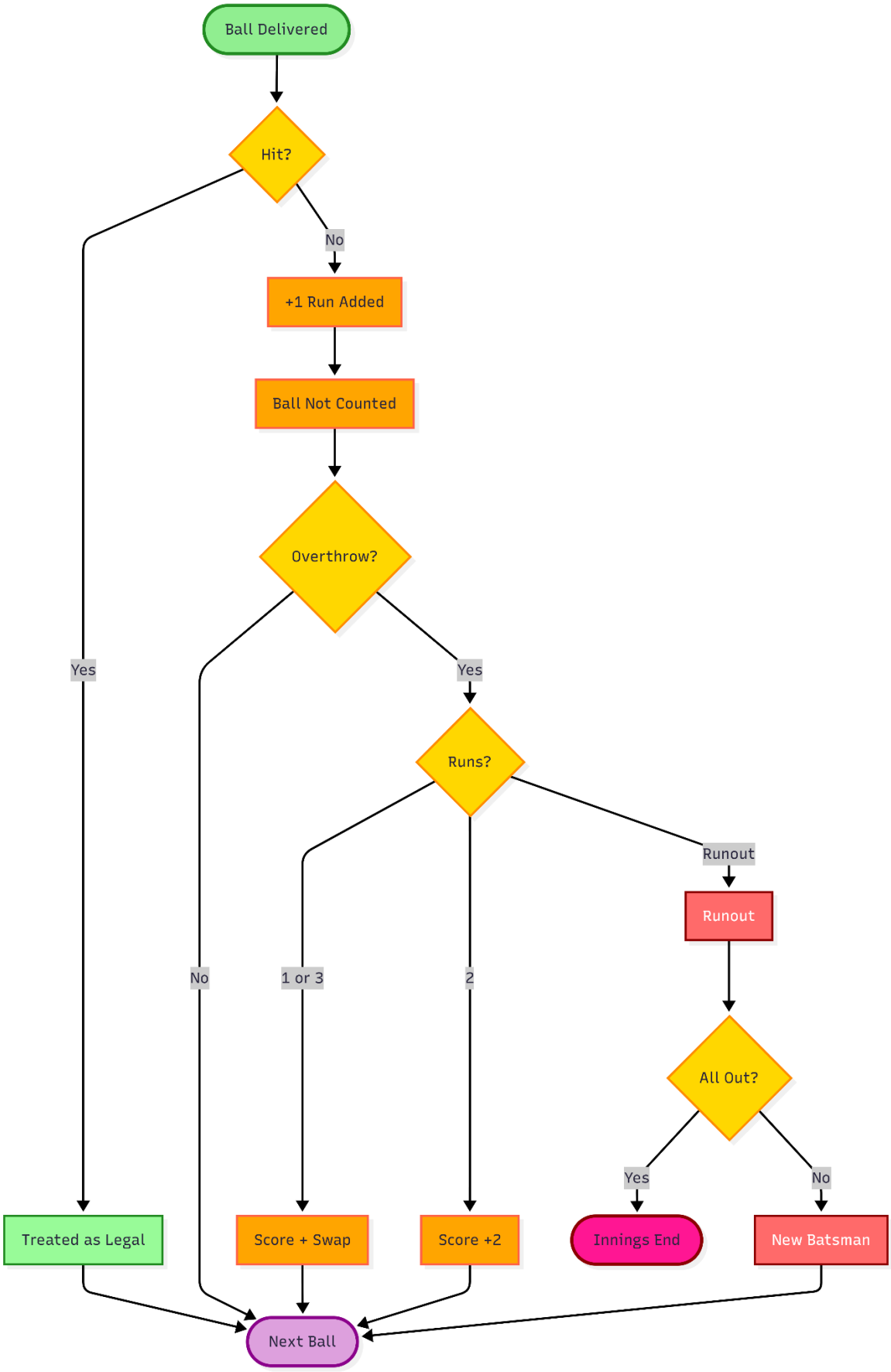
A bowler may deliver three kinds of balls : Legal, Wide, and No ball.

On each ball, the control flow is as follows:

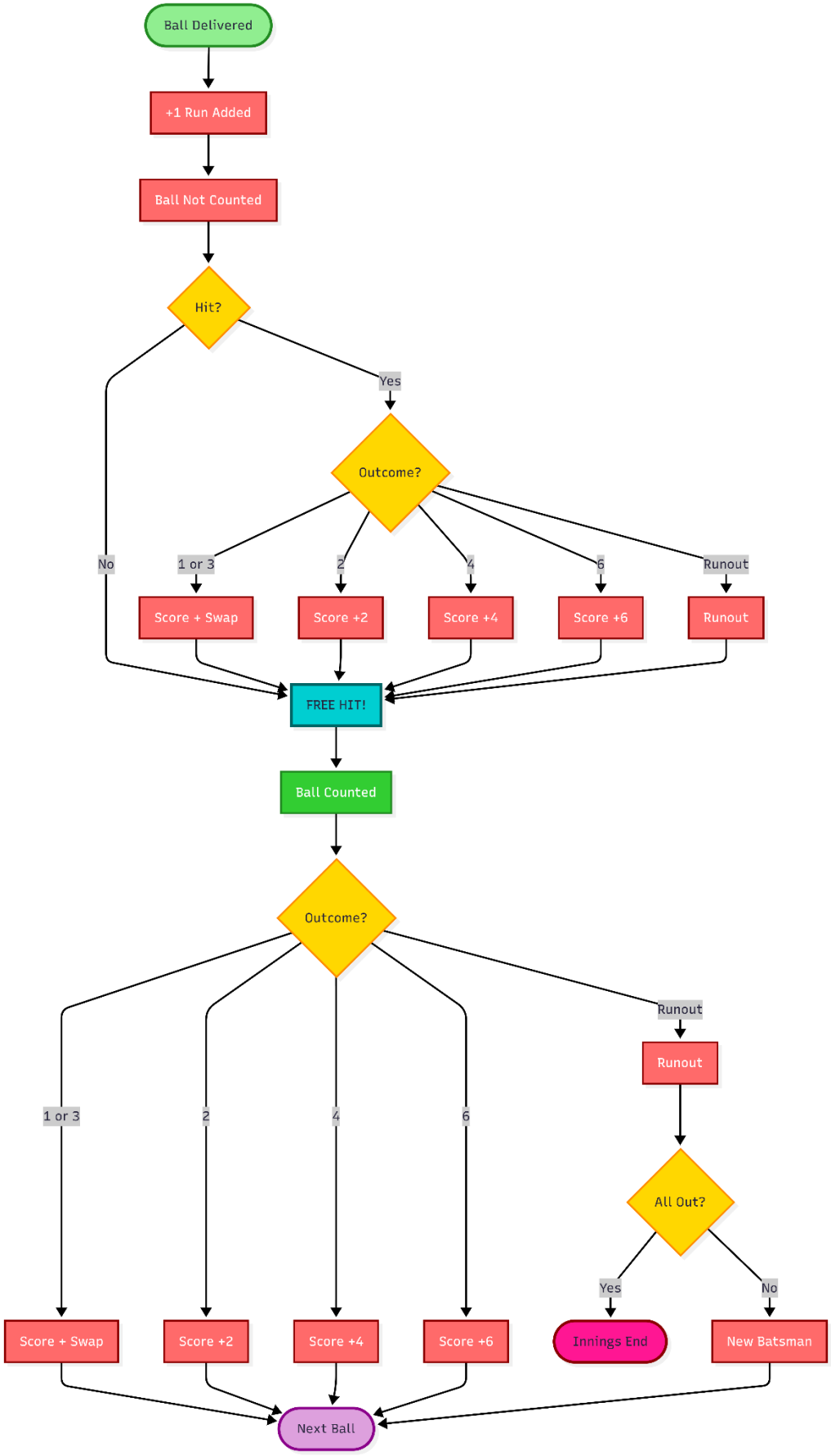
Legal Ball



Wide Ball


Ball Delivered
Hit?
Yes
No
+1 Run Added
Ball Not Counted
Overthrow?
No
Yes
Runs?
1 or 3
2
Runout
Runout
All Out?
Yes
No
Treated as Legal
Score + Swap
Score +2
Innings End
New Batsman
Next Ball

No Ball


Ball Delivered
+1 Run Added
Ball Not Counted
Hit?
Yes
No
Outcome?
1 or 3
2
4
6
Runout
Score + Swap
Score +2
Score +4
Score +6
Runout
FREE HIT!
Ball Counted
Outcome?
1 or 3
2
4
6
Runout
Runout
All Out?
Yes
No
Score + Swap
Score +2
Score +4
Score +6
Innings End
New Batsman
Next Ball

STATISTICAL MODELING OF EACH PLAYER THREAD

To accurately model game simulation, each player thread has associated probabilities which choose a branch of game simulation. Namely, the probabilities are:

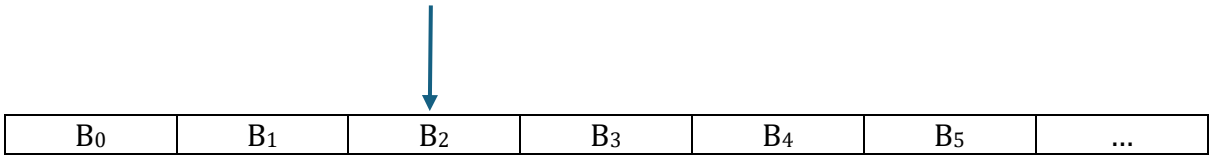
- *Four Probability* – Probability that the current active batsman hits a 4.
- *Six Probability* – Probability that the current active batsman hits a 6.
- *Boundary Probability* – Probability that the current active hits a boundary.
  Note – *Boundary Probability = Four Probability + Six Probability*
- *Legal Ball Probability* – Probability that the current bowler delivers a legal ball on the current delivery.
- *Wide Ball Probability* – Probability that the current bowler delivers a wide ball on the current delivery.
- *No Ball Probability* – Probability that the current bowler delivers a no ball on the current delivery.
- *Run-out Probability* – Probability that the current batsman gets out when facing the next delivery.

SCHEDULING THE PLAYER THREADS

The scheduling for all threads is done at the **User-Level**.

Scheduling the Bowler Threads

The bowler threads follow a **RR scheduling algorithm** for overs 1-15. At the end of each over (6 legal deliveries), the next bowler is scheduled.



RR Scheduling for Bowler Threads

When the intensity of the match is high (last five overs), the scheduler scans for any death-over specialist bowlers. If any are available, they are given priority and scheduled (**Priority scheduling**).

Scheduling the Batsman Threads

We employ two different scheduling strategies for the batsmen:

- **FCFS Scheduling** – Batsmen are scheduled in the standard order as they come. This is similar to real-life cricket.

- **SJF Scheduling** – The "bad" Tail-ender batsmen (ones with a higher probability of getting out and a lower probability of scoring a boundary) are scheduled to bat first. The expected Burst Time of a batsman thread correlates inversely with its Out Probability.

SCHEDULING ANALYSIS - FCFS V/S SJF

We analyze the Wait Time for Middle-Order batsmen (batsmen numbered 4-7) under each scheduling algorithm. Expectantly, the wait time for the Middle-Order batsmen in SJF is significantly shorter than their wait times in FCFS scheduling. Three demonstrations of the same are presented below:

Run 1

```
MIDDLE ORDER WAIT ANALYSIS
Player        FCFS wait time  SJF wait time
4                     14           8
5                     38           9
6                     39          11
7                     52          20
```

Average Wait Time for Middle-Order Batsmen with FCFS scheduling = *35.75 Balls*.

Average Wait Time for Middle-Order Batsmen with SJF scheduling = *12 Balls*.

Run 2

```
MIDDLE ORDER WAIT ANALYSIS
Player        FCFS wait time  SJF wait time
4                     27           8
5                     34          18
6                     38          21
7                     61          29
```

Average Wait Time for Middle-Order Batsmen with FCFS scheduling = *40 Balls*.

Average Wait Time for Middle-Order Batsmen with SJF scheduling = *19 Balls*.

Run 3

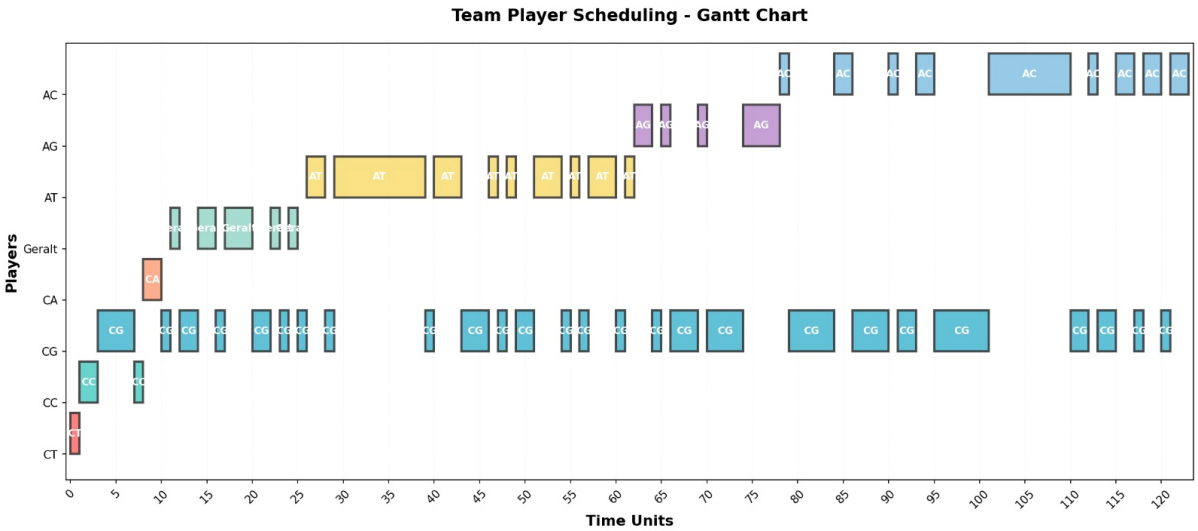
```
MIDDLE ORDER WAIT ANALYSIS
Player        FCFS wait time  SJF wait time
4                     43            7
5                     67           15
6                     77           23
7                     83           31
```

Average Wait Time for Middle-Order Batsmen with FCFS scheduling = *67.5 Balls*.

Average Wait Time for Middle-Order Batsmen with SJF scheduling = *19 Balls*.

ASSUMPTIONS

- We assume that no more than 3 singles are taken by the batsmen in one delivery.
- In the case of a run-out, the active end batsman is always the one that gets out.
- We assume that Hit-the-Ball twice dismissal scenario never occurs.

VISUALIZATIONS

Below we present a Gantt Chart that displays which batsman held the active_end resource at any given time. The x-axis displays the current ball count.